

# Team Git Workflow Documentation

**Fork → Clone → Develop → Pull → Push**

---

## 1 Introduction

This document explains how team members should work on a shared GitHub project.

This workflow is designed for:

- Beginners
- College students
- Team-based academic projects

The goal is:

- Safe collaboration
  - No code conflicts
  - Clean project structure
  - Proper version control
- 

## 2 Project Branch Structure

We follow this branch structure:

```
main      → Production ready code (Only Team Lead controls)
develop   → Active development branch
feature/* → Individual team member work
```

Rules:

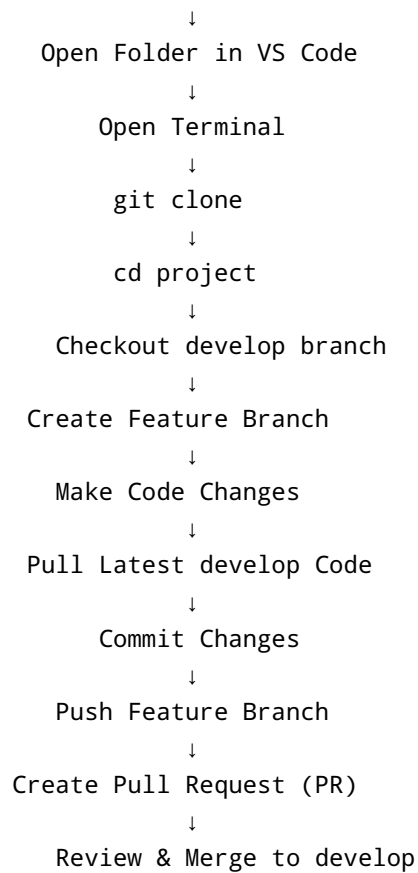
- Never push directly to `main`
  - Never work directly on `develop`
  - Always create a `feature` branch
- 

## 3 Complete Workflow Overview

Team Repository (Original)

↓

Fork



---

## 4 Step-by-Step Detailed Guide

---

### ✓ STEP 1 : Fork the Repository

- 1 . Go to the original team repository on GitHub.
- 2 . Click the **Fork** button.
- 3 . Now a copy of the project is created in your GitHub account.

Now you control your own copy.

---

### ✓ STEP 2 : Open Project Folder in VS Code

- 1 . Open VS Code.
- 2 . Click **File** → **Open Folder**.
- 3 . Select the location where you want to store the project.
- 4 . Click **Open**.

Now VS Code is ready for cloning the repository.

---

### ✓ STEP 3 : Open Terminal in VS Code

Click:

Terminal → New Terminal

OR press:

Ctrl + `

The terminal will open at the bottom of VS Code.

---

### ✓ STEP 4 : Clone Your Fork

Go to your GitHub fork. Click **Code** → **Copy HTTPS URL**.

In VS Code terminal, paste:

```
git clone https://github.com/yourname/project.git
```

Press Enter.

Git will:

- Create a new project folder
- Download all project files

Now move inside the project:

```
cd project
```

---

### ✓ STEP 5 : Checkout develop Branch

Switch to the develop branch:

```
git checkout develop
```

If the develop branch is not available locally:

```
git fetch origin  
git checkout -b develop origin/develop
```

---

## ✓ STEP 6 : Create Your Feature Branch

Never work directly on develop.

Create your own branch:

```
git checkout -b feature/login-page
```

Branch naming examples:

- feature/login-page
- feature/dashboard-ui
- bugfix/api-error
- ui/navbar-design

---

## ✓ STEP 7 : Make Code Changes

Now:

- Add new features
- Fix bugs
- Update UI
- Improve backend logic

Save all files properly.

---

## ✓ STEP 8 : Pull Latest Code Before Pushing

Before pushing your work, always update the develop branch.

```
git checkout develop  
git pull origin develop
```

Now return to your branch:

```
git checkout feature/login-page  
git merge develop
```

This ensures you have the latest team updates.

---

## ✓ STEP 9 : Commit Your Changes

After completing work:

```
git add .  
git commit -m "Added login validation feature"
```

Write clear and meaningful commit messages.

---

## ✓ STEP 10 : Push Your Feature Branch

```
git push origin feature/login-page
```

Now your branch is uploaded to GitHub.

---

## ✓ STEP 11 : Create Pull Request (PR)

- 1 . Go to your GitHub fork
- 2 . Click **Compare & Pull Request**
- 3 . Base branch → develop
- 4 . Add description of changes
- 5 . Click **Create Pull Request**

Now the Team Lead will review your code.

---

## 5 Daily Work Routine for Team Members

Before starting work every day:

```
git checkout develop  
git pull origin develop
```

Then create a new feature branch and start working.

---

## 6 Important Team Rules

- ✗ Do not push directly to main
- ✗ Do not work directly on develop
- ✗ Do not skip pulling latest code

- ✓ Always create feature branch
  - ✓ Always pull before push
  - ✓ Always test your code
  - ✓ Always write proper commit message
- 

## Why This Workflow is Good

- Prevents code conflicts
  - Keeps production safe
  - Makes teamwork easy
  - Maintains clean project history
  - Easy code review process
- 

## Final Summary

The correct team workflow is:

Fork → Clone → Checkout develop → Create Feature Branch → Code → Pull Latest → Co  
→ Create PR → Merge

If every team member follows this process, the project will stay organized and conflict-free.